



Nom i Cognom:	Mohamed Adghir Ettaibi
Enllaç al repositori Git:	https://github.com/adghiretta-alt/punt-partida-26-27

Objectius:

- Aprendre a escriure i llegir arxius de text en Java mitjançant fluxos de dades, manipular fitxers en diversos formats, i aplicar bones pràctiques en la gestió i manipulació de fitxers.

Instruccions:

- Desenvolupa el codi necessari per a cada exercici seguint les especificacions indicades. El codi ha de superar els tests JUnit proporcionats al repositori de punt de partida (**mvn test**).
- Cada exercici té una classe de partida amb el mètode que criden els tests. No canviïs els noms de les classes ni les signatures d'aquests mètodes: si ho fas, els tests no compilaran.

Criteris d'avaluació:

Cada exercici tindrà la mateixa puntuació. Es valorarà:

- Superació dels tests proporcionats.
- Format correcte del codi (indentació i compliment de la guia d'estil de Java).
- Noms clars i descriptius per a mètodes i variables.
- Qualitat general del codi, amb comentaris explicatius quan sigui necessari.
- Les còpies seran penalitzades amb una puntuació de 0.

Entrega:

- Repositori Git privat, compartit amb l'usuari jpala4-ieti, que contingui el codi complet de la pràctica.
- Subdirectori "doc" dins del repositori amb el fitxer memoria.pdf: aquest document en PDF, amb el nom i l'enllaç al repositori emplenats a la capçalera.
- A Moodle només s'ha de lliurar l'URL del repositori.
- El repositori inclou un workflow de GitHub Actions que executa els tests a cada push: en el lliurament final el semàfor ha de quedar en verd.

Recursos i materials:

- JDK de Java versió 21, Maven i Git instal·lats.
- Eina de programació: necessiteu una IDE per programar en Java (VS Code, IntelliJ, o una altra).
- Repositori amb exemples:
<https://github.com/jpala4-ieti/DAM-M0486-RA1-Repositori-Referencia-26-27> (llegeix el fitxer README.md per obtenir instruccions sobre com executar-lo).
- Altres recursos: Podeu utilitzar Google per buscar tutorials que us ajudin a resoldre els exercicis.



Punt de partida de la pràctica:

<https://github.com/jpala4-ieti/DAM-M0486-RA1-PR1.1-Practica-Punt-Partida-26-27>

Resol els exercicis proposats. El directori de treball ha de ser 'data', de la mateixa forma que es fa en el repositori d'exemples. Totes les rutes es construeixen a partir de:

```
String camiBase = System.getProperty("user.dir") + "/data/";
```

Compilació, tests i execució (des de l'arrel del projecte, amb Maven):

- `mvn clean test` → compila el projecte i executa tots els tests JUnit.
- `./run.sh com.project.PR110ReadFile` (Linux/macOS) o `.\run.ps1 com.project.PR110ReadFile` (Windows) → executa una classe concreta.
- Les classes que reben arguments (PR112cat i PR115cp) s'han d'executar passant-los amb `exec.args`, per exemple: `mvn compile exec:java -PrunMain "-Dexec.mainClass=com.project.PR112cat" "-Dexec.args=data/GestioTasques.java"`



Exercici 0: Llegir contingut d'un fitxer i mostrar-lo per pantalla amb números de línia

Completa la classe `PR110ReadFile.java` del repositori de partida, implementant el mètode `llegirIMostrarFitxer(String camiFitxer)`.

Des del mètode, realitza les següents tasques:

- Llegeix el contingut del fitxer `GestioTasques.java`, que es troba dins del directori `data`. La ruta del fitxer serà `System.getProperty("user.dir") + "/data/GestioTasques.java"`.
- Mostra per pantalla tot el contingut del fitxer, afegint el número de línia davant de cada línia de text.
- El format del número de línia serà el següent: cada línia ha d'estar precedida per un número seguit de dos punts (:) i un espai abans del contingut de la línia. Ex.: `1: Aquesta és una línia de prova.`
- Si el fitxer no existeix o es produeix un error de lectura, el programa ha de gestionar l'error i mostrar un missatge adequat, evitant que l'execució es finalitzi de manera abrupta.



Exercici 1: Creació, manipulació i llistat d'arxius

Cal fer-ho usant API NIO.2

Completa la classe `PR111Files.java`, implementant el mètode `gestionarArxius(String camiFitxer)`. El paràmetre és la ruta del directori on cal treballar (des del `main`: `System.getProperty("user.dir") + "/data/pr111"`).

Des del mètode, realitza les següents tasques:

1. Crea una carpeta anomenada `myFiles` dins del directori rebut com a paràmetre (és a dir, a `data/pr111/myFiles` quan s'executa des del `main`). Si les carpetes intermèdies no existeixen, crea-les.
2. Dins d'aquesta carpeta, crea dos arxius: `file1.txt` i `file2.txt`.
3. Mostra un llistat dels arxius dins de la carpeta `myFiles` amb el missatge: "Els arxius de la carpeta són:".
4. Renombra l'arxiu `file2.txt` a `renamedFile.txt`.
5. Elimina l'arxiu `file1.txt`.
6. Torna a mostrar un llistat dels arxius dins de la carpeta `myFiles` amb el missatge: "Els arxius de la carpeta són:". Al final només hi ha de quedar `renamedFile.txt`.



Exercici 2: Simulació de la comanda cat

Completa la classe `PR112cat.java`, implementant el mètode `mostrarContingutArxiu(String rutaArxiu)`.

Des del mètode, realitza les següents tasques:

- El programa rep la ruta d'un arxiu com a argument en executar-lo (el `main` ja la recull i la passa al mètode).
- Si la ruta proporcionada correspon a un **fitxer**:
 - Llegeix el contingut del fitxer en **codificació UTF-8**.
 - Mostra el contingut del fitxer per pantalla.
- Si la ruta proporcionada correspon a una **carpeta**, mostra el missatge: **"El path no correspon a un arxiu, sinó a una carpeta."**
- Si la ruta proporcionada **no correspon a un fitxer existent** o es produeix qualsevol altre error, mostra el missatge: **"El fitxer no existeix o no és accessible."**

Nota: Gestiona adequadament possibles errors de lectura del fitxer i assegura que el programa no falli de manera abrupta. Assegura't que el contingut dels fitxers es llegeix amb la codificació UTF-8. Els dos missatges s'han de mostrar **exactament** com s'indiquen (i res més), perquè els tests els comparen literalment.



Exercici 3: Creació d'un arxiu amb frases de Matrix (UTF-8)

Desenvolupa dos programes que escriguin les següents frases en l'arxiu **frasesMatrix.txt**:

- "I can only show you the door"
- "You're the one that has to walk through it"

Requisits generals:

- El fitxer **frasesMatrix.txt** es trobarà dins el directori **data**, situat a la ruta del projecte. Utilitza **UTF-8** com a codificació per escriure i llegir l'arxiu.
- Cada frase s'escriu en una línia pròpia i **totes les línies, també l'última, acaben amb un salt de línia (\n)**. No s'ha d'afegir cap línia en blanc addicional. El contingut exacte del fitxer després d'executar la versió que sobreescriu és: "I can only show you the door\nYou're the one that has to walk through it\n".
- La ruta del fitxer es pot definir com:
String camiFitxer = System.getProperty("user.dir") + "/data/frasesMatrix.txt";

Requisits específics:

1. **PR113sobreescriu.java** (mètode **escriureFrases(String camiFitxer)**): Aquesta versió sobreescriurà l'arxiu **frasesMatrix.txt** cada vegada que s'executi.
 - Escriu les frases a l'arxiu de manera que qualsevol contingut anterior es perdi.
 - Recorda que l'última frase també ha d'acabar amb un salt de línia (el fitxer tindrà 2 línies).
2. **PR113append.java** (mètode **afegirFrases(String camiFitxer)**): Aquesta versió afegeix les frases al final de l'arxiu **frasesMatrix.txt** sense esborrar el contingut anterior (si el fitxer no existeix, el crea).
 - Cada execució afegeix les dues frases al final, cadascuna acabada amb un salt de línia. Després de dues execucions sobre un fitxer buit, el fitxer ha de tenir exactament 4 línies i acabar amb un salt de línia.
 - Comprova que el contingut existent es manté i que es van afegint correctament les noves línies.



Exercici 4: Generació d'un arxiu amb números aleatoris (UTF-8)

Cal fer-ho usant API NIO.2

Completa la classe `PR114linies.java`, implementant el mètode `generarNumerosAleatoris(String camiFitxer)`.

- El fitxer es guardarà dins del directori **data**, situat a la ruta del projecte (`String camiFitxer = System.getProperty("user.dir") + "/data/numeros.txt";`)
- El programa ha de generar exactament **10 números aleatoris**, cada un en una línia separada.
- Utilitza la codificació **UTF-8** per escriure els números en el fitxer.
- Cada número ha de ser un valor enter aleatori generat entre 0 i 99 (ambdós inclosos).
- Assegura't que cada número estigui separat per un **salt de línia**.
- En aquest cas, després del darrer número **no hi ha cap salt de línia**: el fitxer acaba amb l'últim dígit. Tingues en compte que `Files.write(path, llista)` afegeix un salt després de cada línia, també de l'última; fes servir `Files.writeString()` o un `BufferedWriter`.

Exercici 5: Simulació de la comanda cp

Cal fer-ho usant API NIO.2

Completa la classe `PR115cp.java`, implementant el mètode `copiarArxiu(String rutaOrigen, String rutaDesti)`, que simuli la funcionalitat de la comanda Unix **cp** per copiar fitxers de text.

1. **Arguments del programa:**
 - L'usuari haurà de passar **dues rutes** com a arguments (el `main` ja les recull i les passa al mètode):
 - El primer argument serà la **ruta de l'arxiu origen** que es vol copiar.
 - El segon argument serà la **ruta de destinació** on es vol guardar la còpia.
2. **Funcionalitat:**
 - **Verificació de l'existència del fitxer:**
 - El programa ha de comprovar que el fitxer d'origen existeix i que és un fitxer regular (no una carpeta). Si no ho és, ha de mostrar un missatge d'error i **no** ha de crear el fitxer de destinació.
 - **Còpia del contingut:**
 - El contingut del fitxer origen es copiarà línia a línia a l'arxiu de destinació.
 - **Codificació UTF-8:**
 - Totes les operacions de lectura i escriptura han de fer-se amb codificació UTF-8.
 - **Salt de línia final:**
 - El fitxer de destinació ha de ser una còpia exacta de l'origen: les línies en blanc intermèdies es conserven i, si l'origen acaba amb un salt de línia, el destí també ha d'acabar-hi (i si no, tampoc).
3. **Advertències:**



- Si l'arxiu de destinació ja existeix, el programa ha de mostrar un missatge d'advertència indicant que l'arxiu serà sobreescrit.
4. **Comportament final:**
- El programa ha de mostrar un missatge indicant que la còpia s'ha realitzat correctament, o un missatge d'error si la còpia falla.

Exemple d'ús (des de l'arrel del projecte):

```
mvn compile exec:java -PrunMain "-Dexec.mainClass=com.project.PR115cp"
"-Dexec.args=data/origen.txt data/desti.txt"
```




Exercicis d'ampliació (opcional)

Nom del programa `PR11_ampliacio.java`.

Genera un fitxer de text amb emojis en el directori 'data' fent servir aquesta llibreria (cal afegir-ne la dependència al `pom.xml`):

<https://github.com/vdurmont/emoji-java>

El contingut exacte del fitxer generat el pots decidir tu mateix. S'ha d'anomenar `"emojis.txt"`